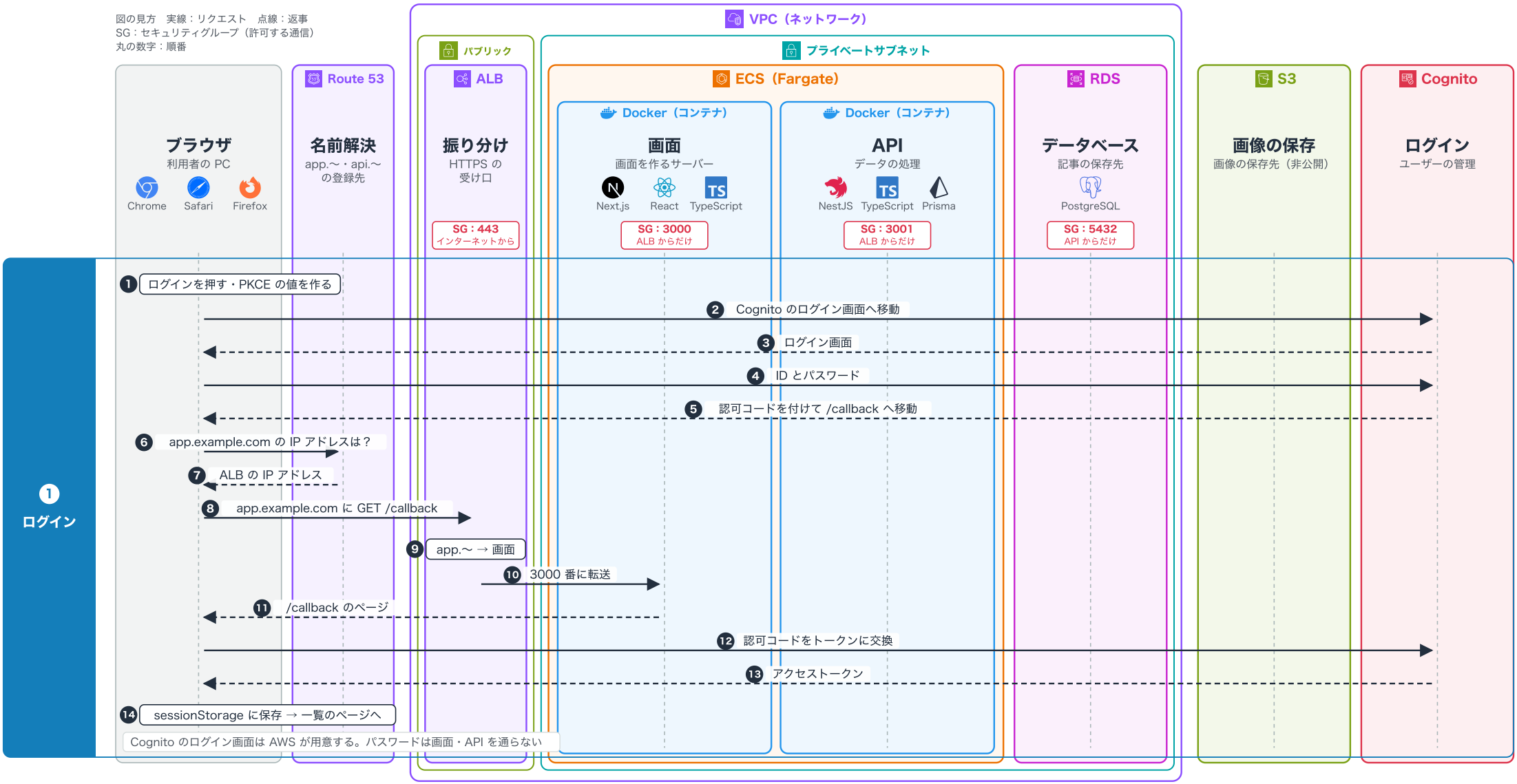
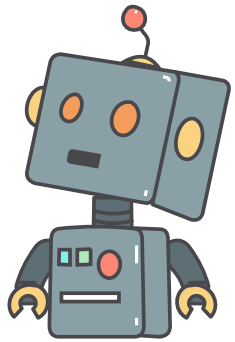
mini-blog のデータの流れ　① ログインする



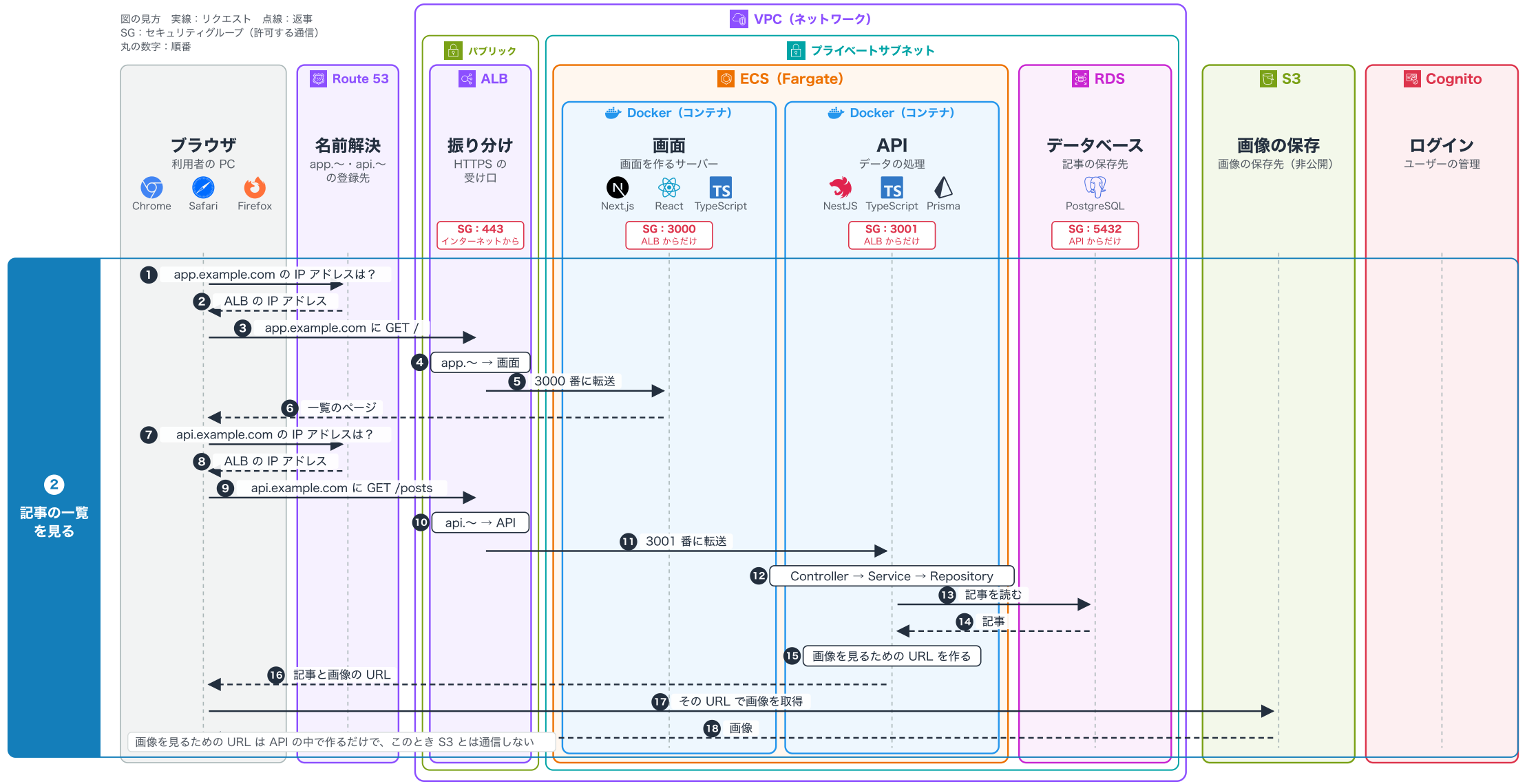
実装と設定

番号	手順	作るもの・設定	参照
1・2	ログインのボタンで PKCE の値を作り、Cognito のログイン画面へ移動する	コード：lib/auth.ts・components/AuthButton.tsx	4-2-8・4-2-10
3・4	ログイン画面は Cognito が用意する。ユーザーは前もって作っておく	設定：Cognito のユーザープール・アプリクライアント・ドメイン	4-1-6・4-1-8
5	ログインのあとに戻る先を、Cognito に登録しておく	設定：Cognito のコールバック URL に https://app.〜/callback を登録	4-1-6・5-1-2
6・7	app.〜 の名前を ALB に向ける	設定：Route 53 のレコード	5-1-7
8・9	HTTPS で受けて、ホスト名で振り分ける	設定：ALB のリスナーとルール、ACM の証明書	5-1-7
10	ALB から画面のコンテナに 3000 番で届ける	設定：セキュリティグループ、ECS のサービス	5-1-4・5-1-10
11〜14	/callback のページが、認可コードをトークンに交換して保存する	コード：app/callback/page.tsx・lib/auth.ts	4-2-9

どの流れでも使うもの：ECR（コンテナのイメージの置き場所）・IAM のタスク実行ロール・CloudWatch Logs（ログ）・VPC と NAT ゲートウェイ・Terraform（AWS の設定を全部コードで作る）　→ 5-1



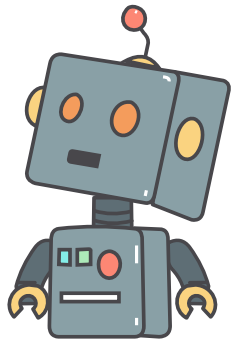
mini-blog のデータの流れ　② 記事の一覧を見る



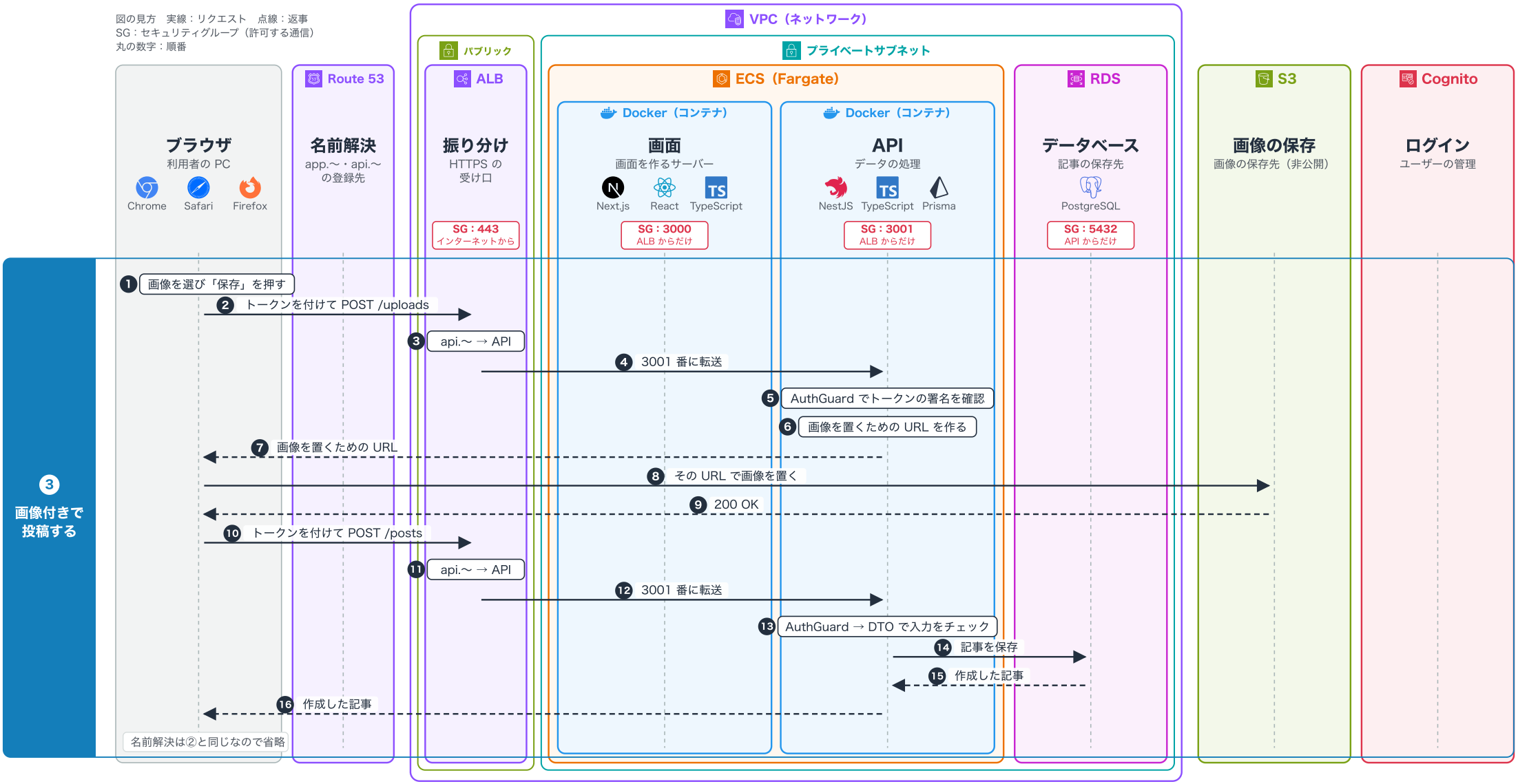
実装と設定

番号	手順	作るもの・設定	参照
1〜8	画面を開くまでは①と同じ。api.〜 の名前も ALB に向ける	設定：Route 53 のレコード、ALB のルール	5-1-7
6・9	一覧のページの JavaScript が、API に記事を取りに行く	コード：app/page.tsx。API の URL は NEXT_PUBLIC_API_URL	3-3-2
9	画面と API はドメインが違うので、API で CORS を許可する	コード：main.ts の enableCors と CORS_ORIGIN	3-3-3
11	ALB から API のコンテナに 3001 番で届ける	設定：セキュリティグループ、ECS のサービス	5-1-4・5-1-10
12	受け付け → 処理 → DB の読み書き、に分けて書く	コード：posts.controller.ts・posts.service.ts・posts.repository.ts	3-2-9〜3-2-11
13・14	Prisma で RDS から記事を読む。接続文字列は起動時に受け取る	コード：schema.prisma・PrismaService。設定：RDS、Secrets Manager	3-2-6・3-2-8・5-1-8
15	画像を見るための URL を作る。その URL で取れるよう、API のロールに S3 を読む権限を付ける	コード：storage.service.ts。設定：IAM のタスクロール	4-2-6・4-2-7・5-1-9
17・18	ブラウザが S3 から画像を直接取る	設定：S3 のバケット	4-1-5

どの流れでも使うもの：ECR（コンテナのイメージの置き場所）・IAM のタスク実行ロール・CloudWatch Logs（ログ）・VPC と NAT ゲートウェイ・Terraform（AWS の設定を全部コードで作る）　→ 5-1



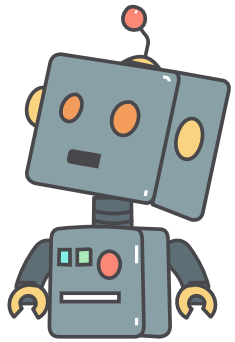
mini-blog のデータの流れ　③ 画像付きで投稿する



実装と設定

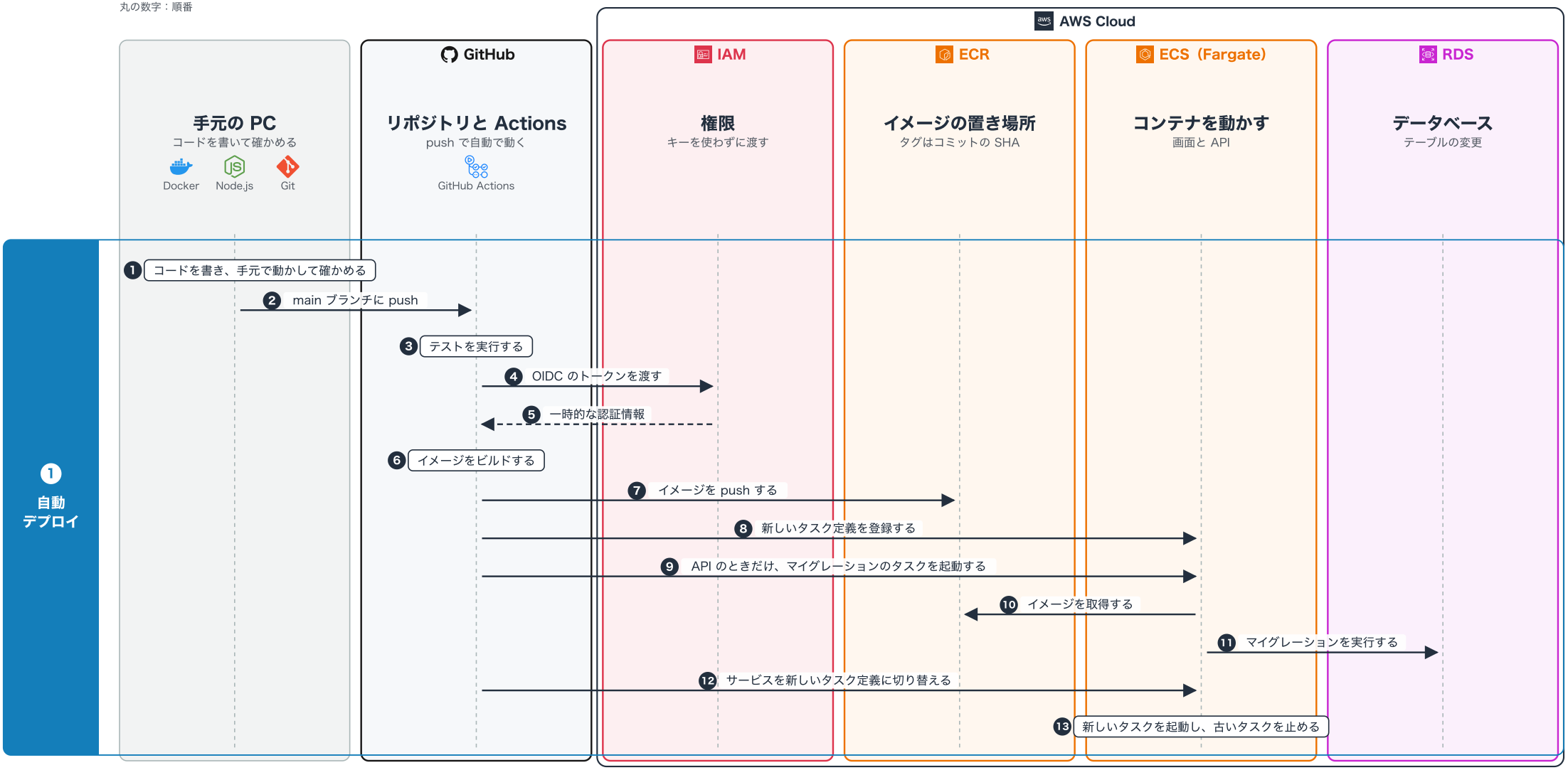
番号	手順	作るもの・設定	参照
1・2	画像を選んで保存すると、まず API に画像を置く URL を頼む	コード：components/PostForm.tsx・lib/upload.ts	4-2-12
2・10	API に送るときに、アクセストークンを付ける	コード：lib/upload.ts・app/posts/new/page.tsx の Authorization ヘッダー	4-2-11
5・13	トークンの署名を確かめる。Cognito の公開鍵は NAT ゲートウェイを通って取りに行く	コード：auth/auth.guard.ts（aws-jwt-verify）	4-2-2・4-2-3
6	画像を置くための URL を作る。その URL で置けるよう、API のロールに S3 に書く権限を付ける	コード：storage.service.ts・uploads.controller.ts。設定：IAM のタスクロール	4-2-6・5-1-9
8・9	ブラウザから S3 に直接置く。app.〜 からの PUT を許可する	設定：S3 の CORS	4-1-5
13	送られてきた内容をチェックする	コード：dto/create-post.dto.ts と ValidationPipe	3-2-12
14	画像のキー（imageKey）も一緒に保存する	コード：schema.prisma に imageKey を足してマイグレーション	4-2-4

どの流れでも使うもの：ECR（コンテナのイメージの置き場所）・IAM のタスク実行ロール・CloudWatch Logs（ログ）・VPC と NAT ゲートウェイ・Terraform（AWS の設定を全部コードで作る）　→ 5-1



mini-blog のデプロイの流れ　① 自動デプロイ

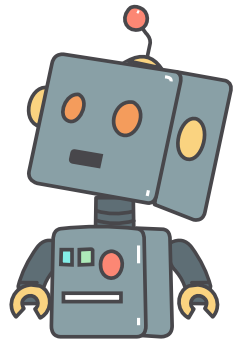
図の見方　実線：リクエスト　点線：返事
丸の数字：順番



実装と設定

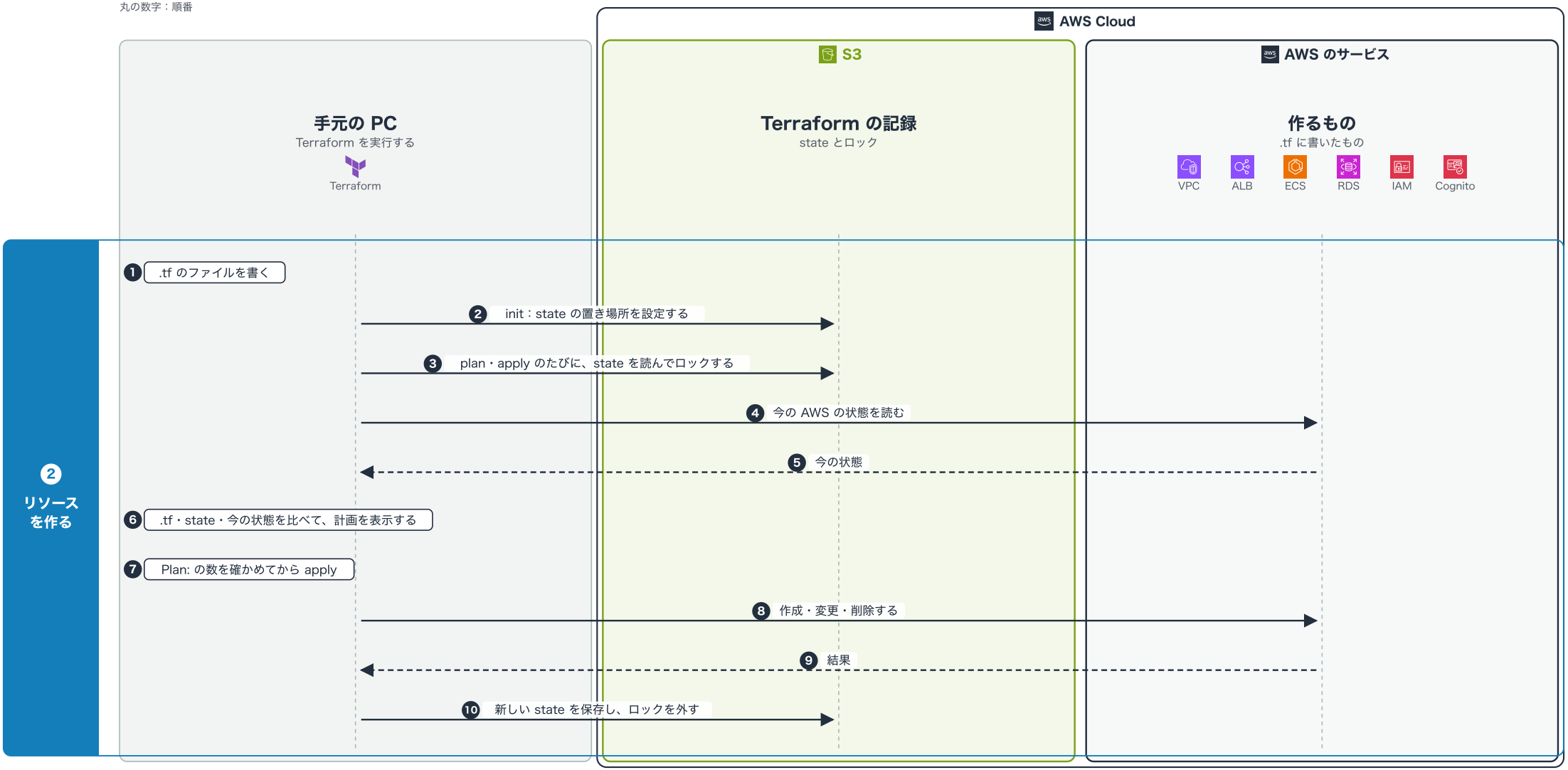
番号	手順	作るもの・設定	参照
1	手元で、Docker のコンテナで動かして確かめる	コード：compose.yaml、backend・frontend の Dockerfile	4-3
2	main ブランチへの push で、workflow が動く	コード：.github/workflows/backend.yml・frontend.yml	5-2-4・5-2-5
3	テストに失敗したら、デプロイしない	コード：workflow の test のジョブ。lint とテスト、画面はビルドで確かめる	5-2-4
4・5	アクセスキーを使わずに、AWS のロールを受け取る	設定：IAM の ID プロバイダと GitHub Actions 用のロール、GitHub の Variables	5-2-2・5-2-3
6・7	イメージをビルドして、コミットの SHA のタグで置く	コード：_deploy.yml。設定：ECR のリポジトリ	5-2-4・5-1-6
8	イメージだけを書き換えたタスク定義を登録する	コード：_deploy.yml。設定：ロールにタスク定義の登録と PassRole の権限	5-2-2・5-2-4
9〜11	API のときだけ、マイグレーションを1回だけのタスクで実行する	コード：_deploy.yml の prisma migrate deploy。設定：RunTask の権限、タスク実行ロール	5-2-4・5-1-9
12・13	サービスを切り替えて、入れ替わりが終わるまで待つ	コード：_deploy.yml の update-service と wait services-stable	5-2-4

prod へのデプロイ：GitHub の Actions で release の「Run workflow」を押すと、同じ手順で動く。release.yml と _PROD の Variables　→ 5-3-4・5-3-5



mini-blog のデプロイの流れ　② AWS のリソースを作る（Terraform）

図の見方　実線：リクエスト　点線：返事
丸の数字：順番



実装と設定

番号	手順	作るもの・設定	参照
1	.tf のファイルに、作るリソースを書く	コード：infra/*.tf（vpc.tf・alb.tf・ecs.tf・rds.tf など）	4-1-4・5-1
2	state の置き場所を設定する	設定：state のバケットは最初に手で作る。env/dev.s3.tfbackend	4-1-3・4-1-4
3	ロックは S3 だけで行う	設定：backend の use_lockfile	4-1-4
4〜6	計画を作って表示する	コマンド：terraform plan -var-file=env/dev.tfvars -out=dev.tfplan	5-1-2
7〜9	計画のとおりに作成・変更・削除する	コマンド：terraform apply dev.tfplan	5-1-2
10	state は S3 に保存する。git には入れない	設定：.gitignore に tfstate・tfplan	3-1-6

dev と prod：同じコードで、env/dev.tfvars・prod.tfvars と env/dev.s3.tfbackend・prod.s3.tfbackend を切り替える　→ 5-3-1

